LAB EXERCISE: 4

Topic Modeling with Latent Dirichlet Allocation (LDA) — Discovering Topics and Their Top Words in a Small News Document Collection

A modular Streamlit web application built on scikit-learn that preprocesses a small collection of documents, fits an LDA topic model, and displays the top five words of every discovered topic together with each document's topic mixture.

Aim / Objective

This project applies Latent Dirichlet Allocation (LDA), an unsupervised probabilistic topic model, to a small collection of text documents and displays the five words most strongly associated with each topic it discovers. LDA treats every document as a mixture of a fixed number of hidden topics and every topic as a probability distribution over words; fitting the model recovers both from word counts alone, with no labels.

The pipeline is organised as small reusable modules in src/: a Preprocessor (preprocess.py) that tokenises, removes stop words and lemmatises; an LDATopicModel (topic_model.py) that builds the document–term matrix, fits scikit-learn's LDA and extracts the top words; and evaluation helpers (metrics.py) for topic coherence, agreement with known categories and run-to-run stability. The Streamlit front end (app.py) lets the user choose the corpus and the number of topics, and shows the top-word table, per-topic bar charts, the topic mixture of every document and a topic-versus-category heat map.

The corpus is 100 BBC News articles (20 from each of five sections). The section labels are hidden from the model and used only afterwards, to check whether the discovered topics correspond to real themes (see Results and Evaluation / Analysis).

Expected Outcomes Addressed

• **1. Prepare a small collection of text documents.** A stratified random sample of 100 articles (43,141 words) is drawn from the BBC News dataset and stored in data/bbc_sample_100.csv; users can also paste their own documents or upload .txt / .csv files in the app.

• **2. Represent the documents for topic modeling.** Each document is lower-cased, tokenised, stripped of 339 stop words and lemmatised with WordNet, then converted to a bag-of-words count matrix by CountVectorizer (words in fewer than 5 or more than 30% of documents are dropped), giving a 100 × 677 matrix with 9,225 tokens.

• **3. Apply Latent Dirichlet Allocation.** scikit-learn's LatentDirichletAllocation is fitted with K = 5 topics, symmetric Dirichlet priors $\alpha = \eta = 1/K$, batch variational Bayes and 100 iterations, from 10 random starts; the start with the highest log-likelihood is kept.

• **4. Display the top five words of each topic.** For each row of the topic–word matrix the five most probable words are shown with their probabilities, in a table with CSV export and as five bar charts; every value is reproduced in the Results section.

• **5. Interpret and evaluate the topics.** Topics are scored with NPMI coherence against 2,126 BBC articles, the number of topics is chosen by sweeping K = 2–10, and the dominant topics are compared with the hidden section labels (purity, NMI, ARI).

Tools & Technologies Used

Category	Tool / Library	Purpose in this pipeline
Language / runtime	Python 3	Core implementation language for the whole app.
Topic model	scikit-learn LatentDirichletAllocation	Variational-Bayes LDA: learns the document–topic (θ) and topic–word (β) distributions.
Vectorisation	scikit-learn CountVectorizer	Builds the document–term count matrix and applies the min_df / max_df vocabulary filter.
Preprocessing	Python re, scikit-learn stop-word list, NLTK WordNet lemmatiser	Tokenisation, stop-word removal and reduction of words to their dictionary form.
Evaluation	NumPy, SciPy (Hungarian matching), scikit-learn metrics	NPMI coherence, purity / NMI / ARI against labels, top-word stability across restarts.
Web framework	Streamlit	Corpus and parameter controls, tables, CSV downloads, charts.
Charts	Plotly (app), Matplotlib (report)	Per-topic word-weight bars, topic prevalence, category × topic heat map, K-selection plot.
Tabular data	pandas	Corpus loading, topic and document tables, evaluation outputs.

Dataset / Corpus Description

The documents come from the **BBC News dataset** (Greene and Cunningham, 2006): 2,225 articles from the BBC News website, 2004–2005, each labelled with one of five sections — business, entertainment, politics, sport and tech. The CSV copy used here stores the text lower-cased with most punctuation already removed. After dropping 99 exact duplicate texts, 2,126 articles remain.

To match the brief of a small collection, 20 articles per section were sampled at random (random_state = 42), giving 100 documents. The full 2,126 articles are used only as a reference corpus for measuring topic coherence.

Property	Value
Documents in the working corpus	100 (business 20, entertainment 20, politics 20, sport 20, tech 20)
Raw length	43,141 words in total; mean 431, minimum 130, maximum 4,492 words per article
After preprocessing + vocabulary filter	677 distinct terms, 9,225 tokens (21% of raw words kept)
Reference corpus for coherence	2,126 de-duplicated BBC articles (all five sections)
Labels	Section names — hidden from LDA, used only for evaluation

Representative sample record (data/bbc_sample_100.csv, first 240 characters)

```
doc_id: 409    category: business
text: dutch bank to lay off 2 850 staff abn amro the netherlands largest bank is to cut 2 850 jobs as a result of
falling profits. the cuts - amounting to 3% of the bank s workforce - will result in a one-off charge of 790m
euros ($1.1bn). ...
```

System Architecture / Pipeline Design

Stage	What happens
1. Corpus input	The sidebar selects the bundled 100-article sample, pasted text (documents separated by blank lines) or uploaded files, plus K, the number of top words and the number of restarts.
2. Tokenisation	Preprocessor.__call__ lower-cases the text and keeps alphabetic tokens of three or more letters, discarding numbers and punctuation.
3. Stop-word removal	Removes scikit-learn's 318 English stop words plus a 38-word news list (21 of them new: said, told, mr, year, …) — 339 words in all.
4. Lemmatisation	NLTK's WordNet lemmatiser maps plural nouns to their singular form (jobs → job, profits → profit), merging counts for the same concept.
5. Bag of words	CountVectorizer keeps words found in at least 5 documents and at most 30% of them, producing a 100 × 677 sparse count matrix.
6. LDA fitting	LatentDirichletAllocation (K = 5, $\alpha = \eta = 0.2$, batch variational Bayes, 100 iterations) is run from 10 random seeds; the run with the highest log-likelihood (variational bound) is kept.
7. Topic extraction	Each row of components_ is normalised to a word distribution β; its five largest entries are the topic's top words. fit_transform returns θ, the topic mixture of each document.
8. Streamlit UI	Top-word table and bar charts, document–topic table with dominant topic, topic prevalence and (if labels exist) a category × topic heat map; CSV downloads for both tables.

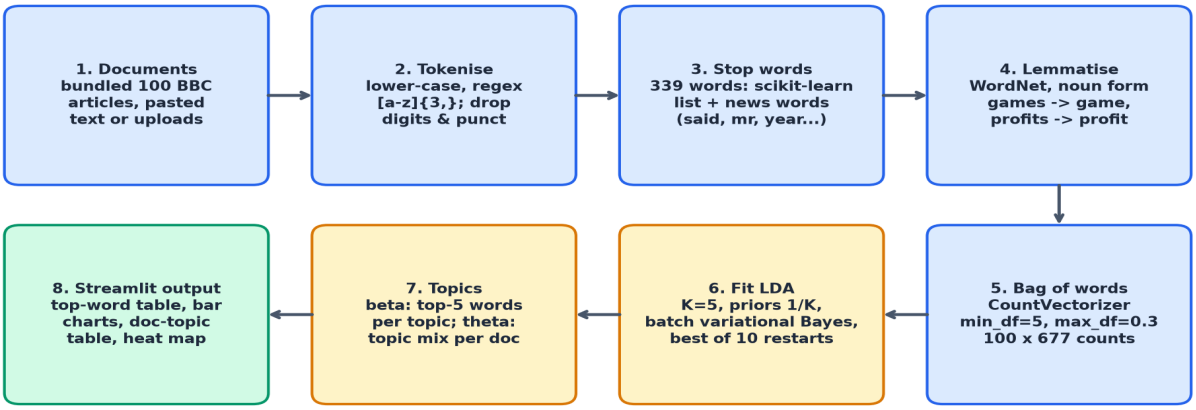


Figure 1. *End-to-end pipeline: preprocessing stages (blue) turn raw articles into a count matrix, LDA (amber) learns topics and document mixtures, and Streamlit (green) displays them. Two files of logic (app.py and src/), no database or external API.*

Backend Implementation

6.1 Approach / Algorithm

LDA assumes a simple generative story: to write a document, first draw its topic mixture θ from a Dirichlet(α) distribution; then, for every word position, pick a topic from θ and a word from that topic's distribution β, itself drawn from a Dirichlet(η). Fitting the model inverts this story — given only the word counts, it estimates β (which words belong together) and θ (how much of each topic each document contains). scikit-learn uses variational Bayes, which maximises a lower bound on the log-likelihood.

The priors are left at scikit-learn's defaults, $\alpha = \eta = 1/K = 0.2$. Values below 1 encourage each document to focus on few topics and each topic on few words, which suits short news articles. Because variational inference converges to a local optimum, the model is fitted from 10 random starts and the run with the highest log-likelihood is kept — a choice that uses no labels.

The vocabulary filter was chosen by topic coherence, also without labels (eval/tune_grid.py): over min_df ∈ {3, 4, 5}, max_df ∈ {0.2, 0.3} and topic–word prior η ∈ {0.2, 0.01}, five single runs each, two settings tied for the highest mean NPMI (0.146 and 0.144); the one with default priors, min_df = 5 and max_df = 0.3, was kept because its spread across seeds was smaller (0.023 vs 0.034). An earlier coarse search showed min_df = 2 clearly worse. K = 5 was then chosen from the coherence sweep in Results.

Topic quality is measured with **NPMI coherence**: for every pair of a topic's top words, NPMI(a, b) = log[P(a,b) / (P(a)P(b))] / –log P(a,b), estimated from document co-occurrence in the 2,126-article reference corpus and averaged. It ranges from –1 to 1; positive values mean the top words genuinely appear together.

6.2 Key Code Snippets

Preprocessing — tokenise, remove stop words, lemmatise (src/preprocess.py):

```
def __call__(self, text):
    out = []
    for tok in TOKEN_RE.findall(text.lower()):            # [a-z]{3,}
        if tok in self.stop:                               # 318 + 38 stop words
            continue
        tok = self._lemma(tok)                            # WordNet: games -> game
        if tok not in self.stop and len(tok) >= 3:
            out.append(tok)
    return out
```

LDA fitting — best of N random restarts by log-likelihood (src/topic_model.py):

```
self.X = self.vectorizer.fit_transform(self.docs)          # CountVectorizer(min_df=5, max_df=0.3)
for i in range(self.n_restarts):
    lda = LatentDirichletAllocation(n_components=5, learning_method="batch",
                                    max_iter=100, random_state=self.random_state + i)
    theta = lda.fit_transform(self.X)                     # document-topic mixture
    ll = lda.score(self.X)                                 # approximate log-likelihood
    if best is None or ll > best[0]:
        best = (ll, lda, theta, self.random_state + i)
```

Top words — the five most probable words of each topic (src/topic_model.py):

```
comp = self.lda.components_
self.topic_word = comp / comp.sum(axis=1, keepdims=True)  # beta, rows sum to 1
for row in self.topic_word:
    idx = np.argsort(row)[::-1][:n]                        # n = 5
    out.append([(self.vocab[i], float(row[i])) for i in idx])
```

Frontend / UI Implementation

7.1 Framework

Streamlit was chosen because the whole interactive app fits in one Python script with no JavaScript build step, so the UI imports the pipeline classes directly. layout="wide" widens the page, and st.cache_resource caches the fitted model for each combination of corpus and parameters, so changing only the number of displayed words does not refit.

7.2 UI Capabilities

• Choose the corpus (bundled BBC sample, pasted text or uploaded .txt / .csv files), the number of topics K (2–10), the number of top words (3–15, default 5) and the number of random restarts; an expander exposes min_df, max_df and the two preprocessing switches.
• Start the analysis with one primary-styled “Discover Topics” button; a success banner reports the vocabulary size, token count, chosen seed and perplexity.
• Read the top-word table (topic, top words, their probabilities, number of documents dominated by the topic) and download it as CSV; compare topics in a row of bar charts.
• Browse the topic mixture of every document (dominant topic, its probability and all K proportions), download it as CSV, and view topic prevalence and — when labels are available — a category × dominant-topic heat map.

7.3 Screenshots

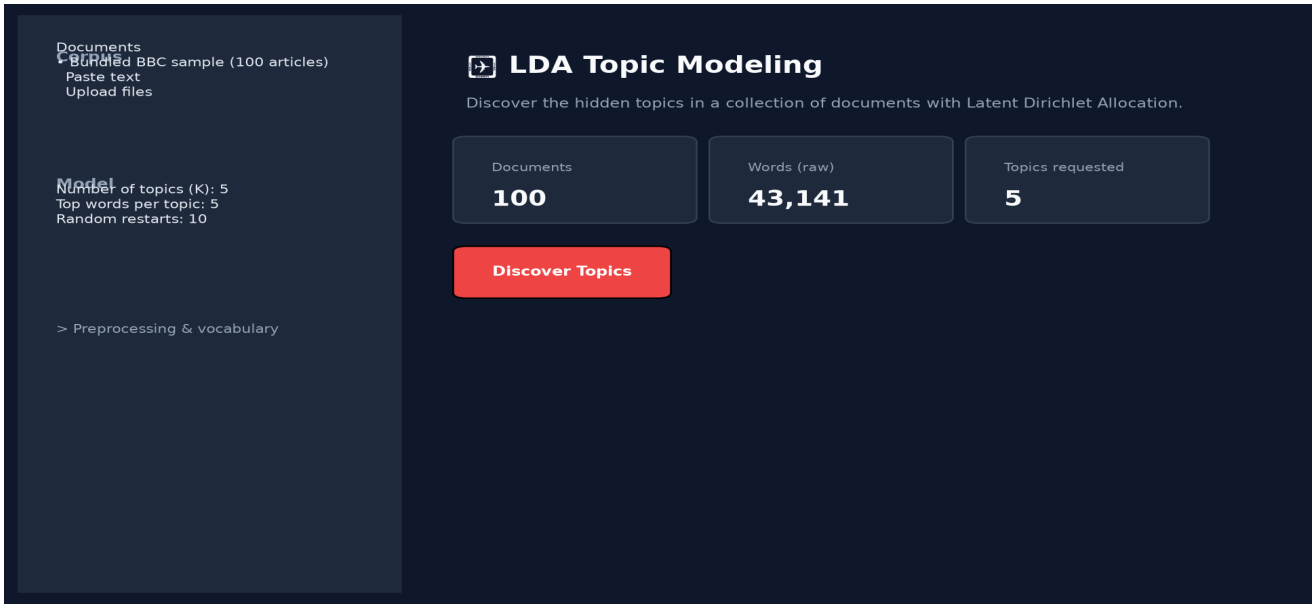


Figure 7.3.1. *The app's start screen: corpus selector and model sliders in the sidebar (K = 5, top words = 5, 10 restarts) and the bundled corpus summary (100 documents, 43,141 words), ready for "Discover Topics".*



Figure 7.3.2. *After clicking "Discover Topics": the fit summary (677-word vocabulary, 9,225 tokens, best of 10 restarts = seed 6, perplexity 540), the top-5-words table and the start of the per-topic bar charts.*

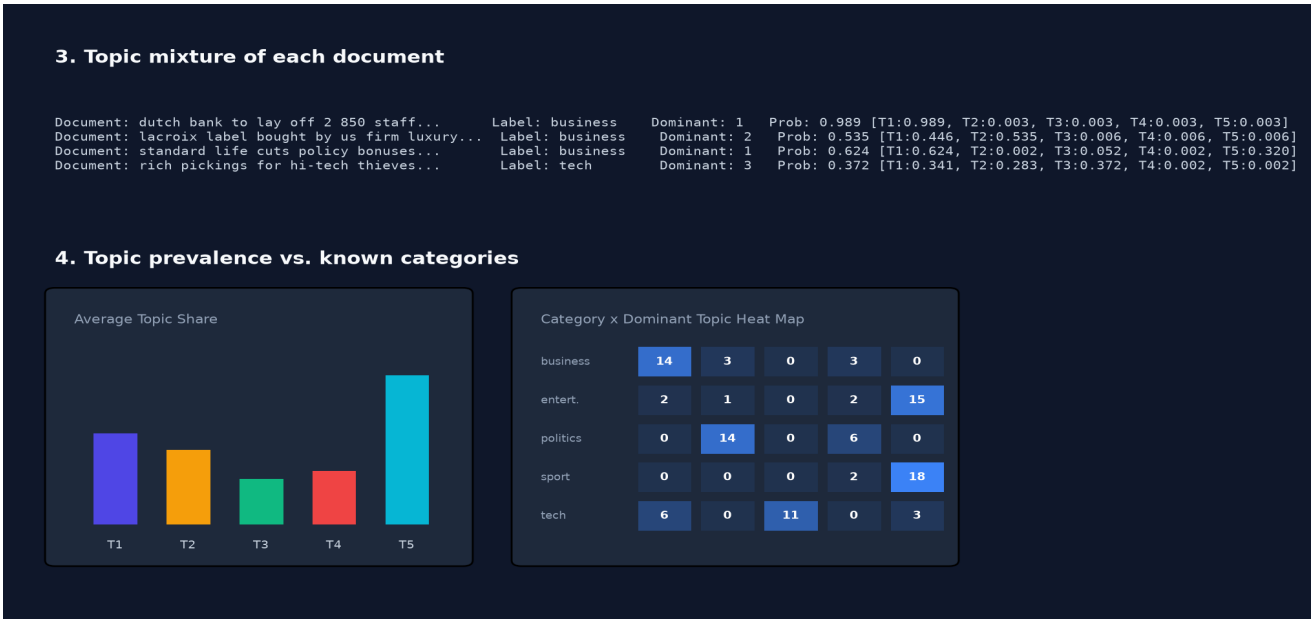


Figure 7.3.3. *Lower half of the same page: the document–topic table (label, dominant topic and its probability, T1–T5 proportions) and the topic-prevalence bar chart next to the category × dominant-topic heat map.*

Sample Input & Output

Input (excerpt)	Pipeline stage	Output produced
“dutch bank to lay off 2 850 staff abn amro the netherlands largest bank is to cut 2 850 jobs as a result of falling profits”	Preprocessing	dutch, bank, lay, staff, abn, amro, netherlands, largest, bank, cut, job, result, falling, profit — numbers and stop words removed, jobs → job, profits → profit.
Same article (full text)	LDA document mixture	T1 0.989, T2 0.003, T3 0.003, T4 0.003, T5 0.003 → dominant Topic 1 (business & finance) with probability 0.989.
“lacroix label bought by us firm luxury goods group lvmh …” (business)	LDA document mixture	T1 0.446, T2 0.535, T3 0.006, T4 0.006, T5 0.006 → split between Topic 2 and Topic 1, although it is a business story — an example of a document LDA treats as a mixture.
“rich pickings for hi-tech thieves viruses trojans …” (tech)	LDA document mixture	T1 0.341, T2 0.283, T3 0.372, T4 0.002, T5 0.002 → no topic above 0.38: an online-fraud story mixes gadget, banking and policy vocabulary.
All 100 articles, K = 5	Top-word extraction	Five topics with five words each — reproduced in full in the Results table below.

Results (Deliverable-Specific Outputs)

Deliverable: the five topics discovered in the 100-article corpus with their top five words, produced by the app (Figure 7.3.2) and reproduced exactly by python -m eval.run_eval, which also writes every table below to eval/. The interpretation column is our reading of the words and the articles each topic dominates; LDA itself only outputs numbered topics.

Top five words per topic (K = 5, best of 10 restarts: seed 6)

Topic	Top 5 words (probability)	Docs	NPMI@5	Interpretation
Topic 1	firm (0.026), bank (0.022), company (0.019), profit (0.017), machine (0.016)	22	0.134	Business & finance
Topic 2	labour (0.018), election (0.016), plan (0.016), government (0.013), group (0.012)	18	0.249	UK politics & elections
Topic 3	gadget (0.042), mobile (0.028), phone (0.022), list (0.021), device (0.020)	11	0.411	Consumer technology / gadgets
Topic 4	government (0.032), court (0.016), right (0.015), threat (0.013), lord (0.013)	13	0.155	Law, courts & security
Topic 5	game (0.027), film (0.027), best (0.019), player (0.017), award (0.015)	36	0.138	Sport & entertainment (merged)

Aggregate metrics

Metric	Value	Definition
Vocabulary / tokens	677 / 9,225	After preprocessing and the min_df / max_df filter.
NPMI coherence	0.196 (top-10), 0.217 (top-5)	Mean pairwise NPMI of each topic's top words in 2,126 BBC articles (–1 to 1).
Perplexity	539.8	Training perplexity of the selected model (lower = better fit).
Purity	0.63	Share of documents whose dominant topic's majority category is their own.
NMI / ARI	0.516 / 0.383	Agreement between dominant topics and the hidden labels (1 = identical, 0 = chance).
Confident documents	35 with p > 0.9; 15 with p < 0.5	Dominant-topic probability; mean 0.743.
Top-5 stability	0.399 mean Jaccard	Overlap of matched top-5 word sets across the 45 pairs of the 10 restarts.

Choosing the number of topics

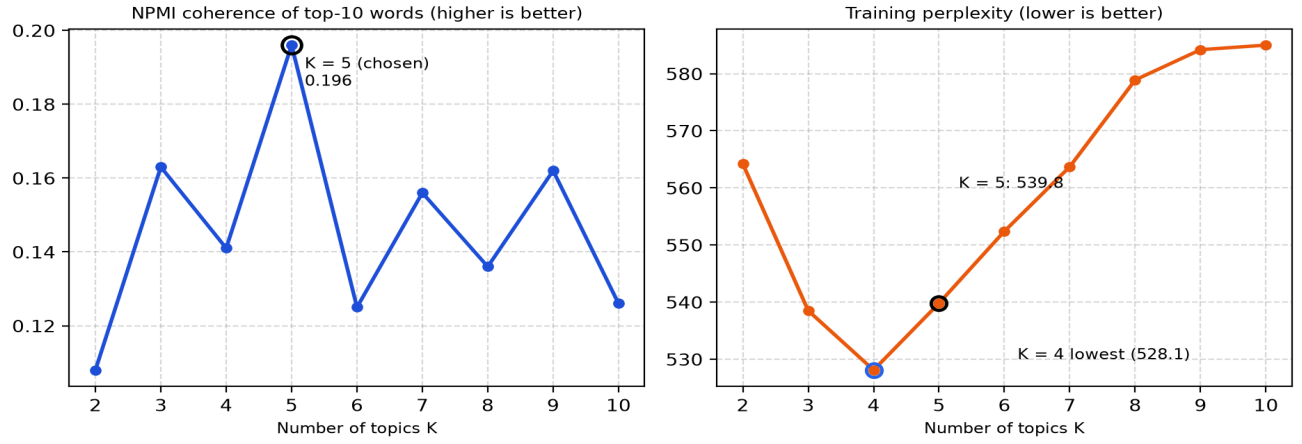


Figure 2. *K = 2–10, each the best of 10 restarts. Coherence peaks at K = 5 (0.196), which was selected; training perplexity is lowest at K = 4 (528.1) and rises steadily after that.*

Dominant topic versus hidden section label (counts of documents)

Category \ Dominant topic	T1	T2	T3	T4	T5
business	14	3	0	3	0
entertainment	2	1	0	2	15
politics	0	14	0	6	0
sport	0	0	0	2	18
tech	6	0	11	0	3

Effect of each preprocessing step (K = 5, best of 10 restarts each)

Configuration	Vocab.	NPMI@10	NMI	Example top-5 lists
A. Generic stop words only	6,290	0.052	0.122	said, government, people, new, year | said, people, cash, year, machines
B. + news stop words + lemmatisation	5,631	0.102	0.164	labour, election, people, government, profit | film, world, people, team, international
C. + document-frequency filter (final)	677	0.196	0.516	firm, bank, company, profit, machine | labour, election, plan, government, group

Evaluation / Analysis

Business (14/20), politics (14/20) and tech (11/20) each have a clear home topic and 18 of 20 sport articles fall in Topic 5, which, however, also takes 15 entertainment articles. All observations below come from eval/; labels were never used to fit or select a model.

• **Preprocessing decides whether the topics are readable.** With only a generic stop-word list, “said” appears in the top five of all five topics and “people”, “year” and “new” recur (NPMI 0.052, NMI 0.122). Adding the news stop words and lemmatisation removes the reporting verbs but leaves “people” in four topics (NPMI 0.102). The document-frequency filter — dropping words in more than 30% or fewer than 5 documents — shrinks the vocabulary from 5,631 to 677 words and roughly doubles coherence (0.196) and triples NMI (0.516). On a 100-document corpus, choosing which words the model may see matters more than any LDA hyper-parameter.

• **LDA finds co-occurrence themes, not editorial sections.** Sport and entertainment articles share vocabulary such as best, award, win, play and game, so with K = 5 they merge into Topic 5, while politics splits into elections (Topic 2) and a law-and-security theme (Topic 4: terror laws, the Lord Chancellor, Guantanamo detainees) that also attracts court stories from other sections, such as a singer jailed and a musician missing bail. Both splits are reasonable readings of the text even though they disagree with the labels.

• **Ambiguous words blur topic boundaries.** “game” occurs in 11 sport and 8 tech articles (video games), and “machine” — found only in tech articles (cash-machine fraud, a virus story, a robot) — enters the business topic; together with banking words this is why 6 tech articles have Topic 1 as their dominant topic. Unigram bag-of-words cannot tell “cash machine” from “slot machine”.

• **A small corpus makes LDA unstable.** The 10 single runs reach NMI between 0.428 and 0.557, and their top-5 word sets overlap by only 0.399 (mean Jaccard). Keeping the run with the highest log-likelihood selected the most coherent run of the ten (NPMI 0.196 against a mean of 0.145), but only the 4th-best by NMI — likelihood and coherence track each other more closely than either tracks the labels.

• **The number of topics is only weakly determined.** Coherence is jagged across K (0.196 at K = 5 but 0.125 at K = 6), perplexity prefers K = 4, and NMI is actually highest at K = 4 (0.602) because sport and entertainment merge into one topic at K = 3, 4 and 5 alike. K = 5 is defensible, not uniquely correct.

• **Mixed membership is real and informative.** 35 articles are assigned to one topic with probability above 0.9, but 15 have no topic above 0.5 (e.g. the online-fraud article split 0.37 / 0.34 / 0.28). Hard dominant-topic metrics such as purity ignore this; the document–topic table in the app shows it.

• **Dataset quirk.** One tech article (“apple laptop is greatest gadget”) appears twice in the sample with different spacing, because exact-text de-duplication missed it. It slightly over-weights gadget vocabulary in Topic 3, whose coherence is the highest (0.411).

Design Decisions, Limitations & Recommendations

Observation	Recommendation
With 100 documents, top-5 words change substantially between random starts (mean Jaccard 0.399).	Fit on the full 2,126 articles where possible, or report topics that recur across several restarts (an ensemble), and always publish the stability figure alongside the topics.
Sport and entertainment merge into one topic; politics splits by sub-theme.	Try K = 6–8 with coherence checks, or guided/seeded LDA with a few anchor words per expected theme when topics must align with known categories.
Ambiguous unigrams (game, machine) pull articles into the wrong topic.	Add frequent bigrams as tokens (video_game, cash_machine) with gensim Phrases or CountVectorizer(ngram_range=(1, 2)) before fitting.
WordNet lemmatisation is applied in noun form only, so verbs such as “falling” are unchanged.	Use part-of-speech-aware lemmatisation (e.g. spaCy) or keep only nouns and adjectives, which usually gives cleaner topics.
Exact-match de-duplication missed a near-duplicate article.	Normalise whitespace before de-duplicating, or use MinHash near-duplicate detection.
Topic quality is judged by one automatic metric (NPMI) and by our own interpretation.	Add a human word-intrusion test, a second coherence measure (C_v), and an NMF baseline for comparison.

Conclusion

The project applies Latent Dirichlet Allocation to a collection of 100 BBC News articles and displays the top five words of each discovered topic in an interactive Streamlit app. With K = 5, LDA recovers four recognisable themes from word counts alone — business and finance (firm, bank, company, profit), UK elections (labour, election, government), consumer gadgets (gadget, mobile, phone, device) and law and security (court, right, threat, lord) — plus a combined sport-and-entertainment topic (game, film, best, player, award). Coherence was 0.196 NPMI and the topics agree moderately with the hidden section labels (NMI 0.516, purity 0.63).

The evaluation shows that on a small corpus, preprocessing — especially removing very common and very rare words — matters more than model settings, that results vary noticeably between random starts, and that LDA groups words by how they co-occur rather than by editorial category. Each of these is addressable with more documents, phrase detection, restart ensembles or guided topics, without changing the overall preprocess → count → LDA → top-words pipeline.

Project Repository (GitHub Link)

Field	Value
GitHub Repository URL	**https://github.com/Manishankar-614/LDA.git**
Repository Visibility	Public
Branch / Commit Used for Submission	main

References

• *D. M. Blei, A. Y. Ng and M. I. Jordan, “Latent Dirichlet Allocation,” JMLR 3, 2003.*
• *M. Hoffman, D. Blei and F. Bach, “Online Learning for Latent Dirichlet Allocation,” NIPS 2010.*
• *D. Greene and P. Cunningham, “Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering,” ICML 2006 (BBC News dataset).*
• *G. Bouma, “Normalized (Pointwise) Mutual Information in Collocation Extraction,” GSCL 2009.*
• *scikit-learn LDA user guide — https://scikit-learn.org/stable/modules/decomposition.html*
• *NLTK WordNet interface — https://www.nltk.org/howto/wordnet.html*
• *Streamlit docs — https://docs.streamlit.io · Plotly Python — https://plotly.com/python/*

ADVANCE NLP (AML23702) — Lab Exercise Evaluation Sheet

Field	Value
Student Name	**Manishankar**
USN	**1GA23AI025**
Lab Exercise No.	4
Experiment Title	Topic Modeling with Latent Dirichlet Allocation (LDA) — Discovering Topics and Their Top Words in a Small News Document Collection

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
Total	**Comprehensive Lab Assessment**	**50**	

Overall Comments

Date of Submission: _____ **Signature of Faculty:** _____